BÖLÜM 6: GÜVENILIRLIK VE HATA TOLERANSI — ÇÖKÜŞ İÇIN TASARLAMAK 💣 9 dk • DERS 24

Parça 24: Sıfır Kesintili Dağıtımlar: Mavi-Yeşil (Blue-Green), Kanarya (Canary) ve Özellik Bayrakları (Feature Flags)

Kullanıcılarınızı ve nöbetçi ekibinizi dehşete düşürmeden günde 50 kez canlıya kod çıkmak.

> 🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**
>
> Bir bordro SaaS şirketinin her Cumartesi gece 02:00'de planlı bakım penceresi vardı. Geliştiriciler bu saatten nefret ederdi. Bir hafta sonu 80 milyon satırlık bir tabloya `NOT NULL` kolon ekleyen bir veritabanı migrasyonu çalıştırıldı. Tablo kilitlendi. Sunucular zaman aşımına düştü. Sabah 05:00'te ekip panikle sürümü geri almaya (rollback) çalıştı ama ara şema değişiklikleri yüzünden geri alma komutları da patladı! Sistem Pazartesi 11:00'e kadar kapalı kaldı. Binlerce şirket çalışanlarının maaşlarını yatıramadı ve şirket 600.000 dolarlık kurumsal sözleşme iptalleriyle sarsıldı.

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

'Bakım penceresi' kavramı 1990'ların antika bir alışkanlığıdır. Modern yazılım mimarisi sıfır kesintili sürekli dağıtımı zorunlu kılar. Kod dağıtmak (sunuculara dosya kopyalamak) ile özellik yayınlamak (kullanıcıların erişimine açmak) birbirinden tamamen koparılmalıdır. **Mavi-Yeşil Dağıtım**, **Kanarya Sürümleri** ve **Özellik Bayrakları (Feature Flags)** ile riski neredeyse sıfıra indirerek gün ortasında canlıya çıkabilirsiniz.

Temel Mimari İlkeler ve Zihinsel Modeller

Sıfır Kesintili Dağıtım Stratejileri

- **Mavi-Yeşil (Blue-Green):** Biri canlı (Mavi), diğeri boşta (Yeşil) iki özdeş ortam. Yeni sürüm Yeşile kurulur, duman testleri yapılır. Yönlendirici bir saniyede trafiği Mavi'den Yeşil'e çevirir. Sorun çıkarsa 1 saniyede geri dönülür.
- **Kanarya (Canary):** Trafiğin %2'si yeni sürüme yönlendirilir. Metrikler 15 dakika izlenir. Sorun yoksa kademeli olarak %10, %50 ve %100'e çıkarılır.
- **Özellik Bayrakları (Feature Flags):** Kod canlıya kapalı olarak atılır. Ürün yöneticisi panodan özelliği belirli kullanıcılara tek tıkla açar.

Genişlet ve Daralt Kalıbı (Expand & Contract)

Veritabanı kolonunu asla tek seferde yeniden adlandırmayın! 3 aşama izleyin:

1. **Genişlet:** Yeni kolonu ekle. Uygulama hem eskiye hem yeniye yazsın.
2. **Göç:** Arka planda eski verileri yeniye kopyala.
3. **Daralt:** Okumaları yeniye al ve bir sonraki sprintte eski kolonu güvenle sil.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Büyük Patlama (Big Bang) Sürümü:** 45 özelliği ve 6 veritabanı migrasyonunu iki ayda bir gecenin köründe tek seferde canlıya almaya kalkışmak.

- **Yıkıcı Veritabanı Değişikliği:** Canlı trafik akarken kolon silmek veya isim değiştirmek; çalışan eski sunucuların anında çökmesi.
- **Özellik Bayrağı Borcu:** Kullanılmış bayrakları koddan temizlemeyip kod tabanını iç içe geçmiş if-else çöplüğüne çevirmek.

> 🎯 **MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** API Gateway veya Ingress seviyesinde Kanarya yönlendirmeyi kur (trafiğin %5'ini yeni pod'lara ver).
> - **2. Adım:** Bütün veritabanı değişikliklerinde 'Genişlet ve Daralt' kuralını zorunlu kıl.
> - **3. Adım:** Kritik her yeni iş özelliğini bir Feature Flag arkasına alarak canlıya al.

Mimari Çıkarımlar ve Hile Tablosu

- Kod canlıya almak heyecanlı bir kriz değil; sıkıcı, rutin bir iş olmalıdır.
- Teknik dağıtım ile işlevsel yayını özellik bayraklarıyla birbirinden ayırın.
- Veritabanı şema değişikliklerini her zaman çalışan eski kodla geriye dönük uyumlu tutun.

> 🎙️ **UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Ekibiniz veritabanı migrasyonlarını nasıl yapıyor? Bir migrasyon yarıda patlasa, sistemi kapatmadan veya veri kaybetmeden geri dönebiliyor musunuz?